1. Importing libraries

In [8]:
```
from pathlib import Path
import warnings
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

sns.set(style="whitegrid")
warnings.filterwarnings("ignore")
```

2. Load data

In [11]:
```
# Use a raw string (r"...") to avoid escape character issues with backslashes
file_path = Path(
    r"C:\Users\red devil\Desktop\PROJECT BI\P5\archive\marketing_AB.csv"
)

# Call pd.read_csv by passing the file_path
df = pd.read_csv(file_path)

# Display the first few rows
df.head()
```

Out[11]:

	Unnamed: 0	user id	test group	converted	total ads	most ads day	most ads hour
0	0	1069124	ad	False	130	Monday	20
1	1	1119715	ad	False	93	Tuesday	22
2	2	1144181	ad	False	21	Tuesday	18
3	3	1435133	ad	False	355	Tuesday	10
4	4	1015700	ad	False	276	Friday	14

In [15]:
```
# Remove unused columns safely (using errors='ignore' prevents KeyErrors if names d
df.drop(columns=['Unnamed: 0', 'user id'], errors='ignore', inplace=True)

# Display first rows
print(df.columns)
df.head()
```

```
Index(['test group', 'converted', 'total ads', 'most ads day',
       'most ads hour'],
      dtype='str')
```

Out[15]:

	test group	converted	total ads	most ads day	most ads hour
0	ad	False	130	Monday	20
1	ad	False	93	Tuesday	22
2	ad	False	21	Tuesday	18
3	ad	False	355	Tuesday	10
4	ad	False	276	Friday	14

3. Basic EDA

In [18]:
```
print("Shape:", df.shape)
print(df.info())

# Summary stats for numeric columns (fixed 'describr' to 'describe')
df.describe().T
```

```
Shape: (588101, 5)
<class 'pandas.DataFrame'>
RangeIndex: 588101 entries, 0 to 588100
Data columns (total 5 columns):
 #   Column         Non-Null Count   Dtype 
---  ------         --------------   ----- 
 0   test group     588101 non-null  str   
 1   converted      588101 non-null  bool  
 2   total ads      588101 non-null  int64 
 3   most ads day   588101 non-null  str   
 4   most ads hour  588101 non-null  int64 
dtypes: bool(1), int64(2), str(2)
memory usage: 23.6 MB
None
```

Out[18]:

	count	mean	std	min	25%	50%	75%	max
total ads	588101.0	24.820876	43.715181	1.0	4.0	13.0	27.0	2065.0
most ads hour	588101.0	14.469061	4.834634	0.0	11.0	14.0	18.0	23.0

In [19]:
```
#unique values
print("Test group:" , df['test group'].unique())
print("Converted:" , df['converted'].unique())
print("Most ads day:", df['most ads day'].unique())
```

```
Test group: <ArrowStringArray>
['ad', 'psa']
Length: 2, dtype: str
Converted: [False  True]
Most ads day: <ArrowStringArray>
['Monday', 'Tuesday', 'Friday', 'Saturday', 'Wednesday', 'Sunday', 'Thursday']
Length: 7, dtype: str
```

4. Data Preprocessing

In [20]:
```
df['converted']
```

Out[20]:
```
0         False
1         False
2         False
3         False
4         False
          ...
588096    False
588097    False
588098    False
588099    False
588100    False
Name: converted, Length: 588101, dtype: bool
```

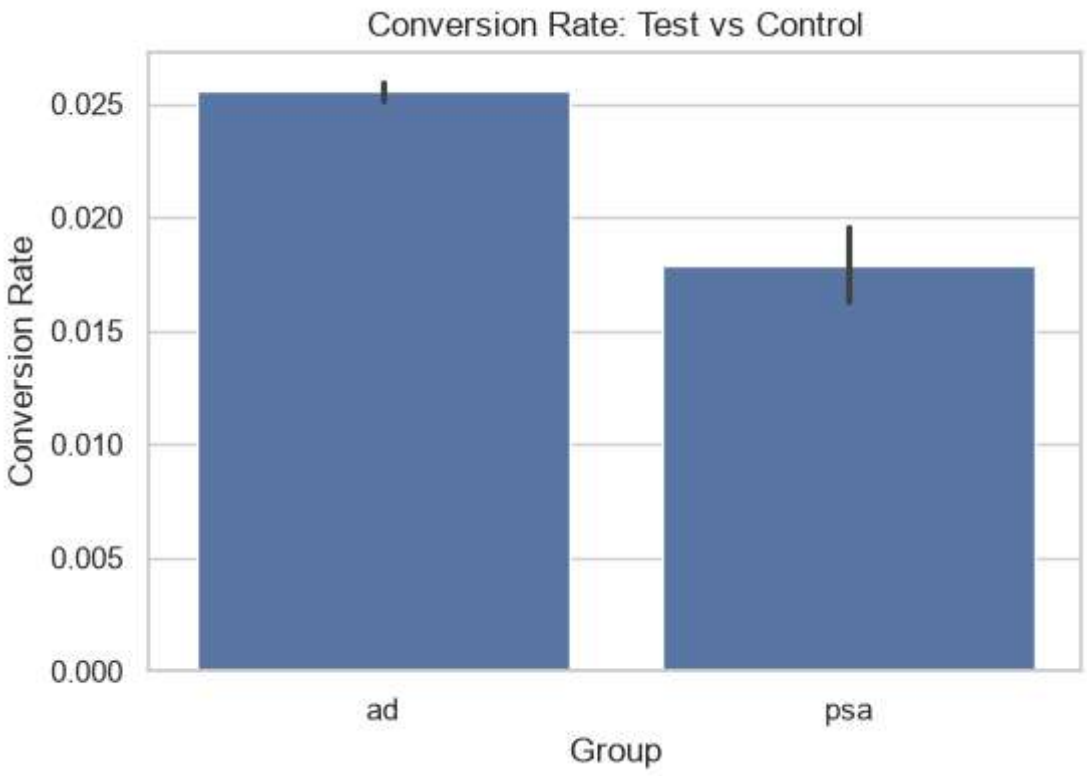
In [21]:
```
# Convert boolean to integer for numerical tests
df['converted'] = df['converted'].astype(int)

df.dtypes
```

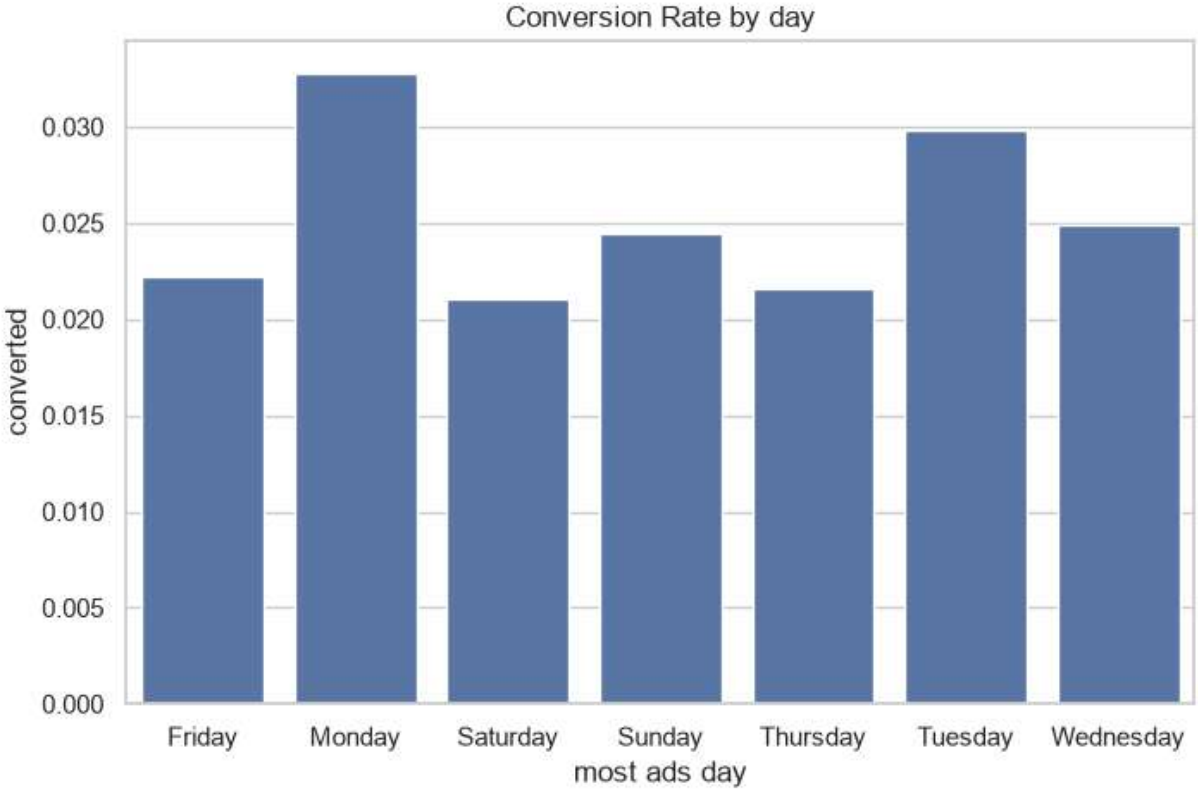
Out[21]:
```
test group         str
converted        int64
total ads        int64
most ads day       str
most ads hour    int64
dtype: object
```

5. Visualizations

In [22]:
```
plt.figure(figsize=(6,4))
sns.barplot(data=df, x='test group', y= 'converted')
plt.title("Conversion Rate: Test vs Control")
plt.xlabel("Group")
plt.ylabel("Conversion Rate")
plt.show()
```

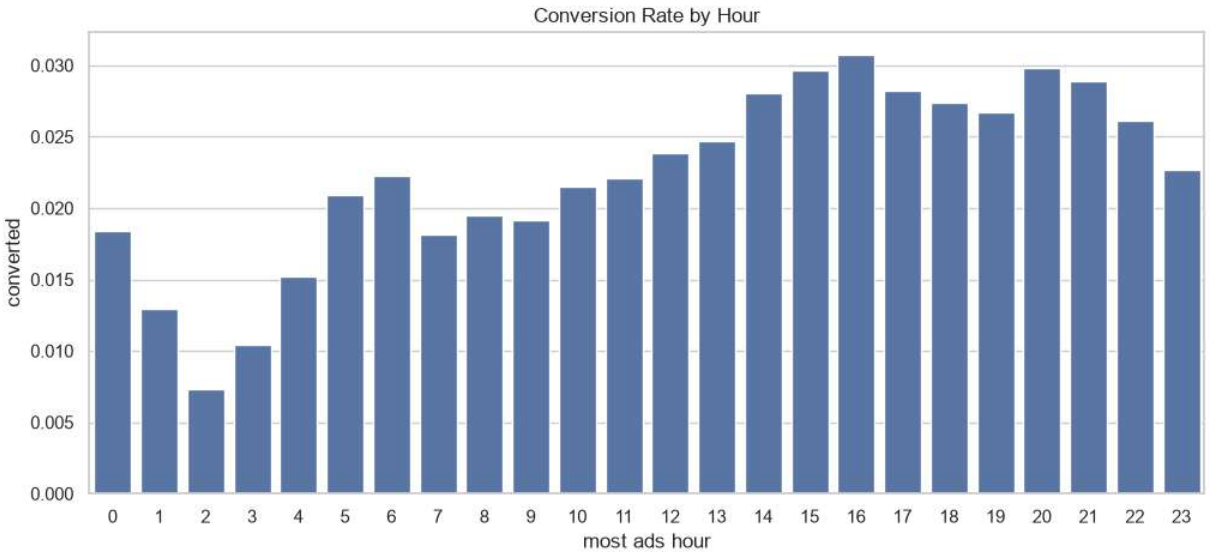


In [23]:
```
day_rate  = df.groupby('most ads day')['converted'].mean().reset_index()
plt.figure(figsize=(8,5))
sns.barplot(data=day_rate, x= 'most ads day' , y= 'converted')
plt.title("Conversion Rate by day")
plt.show()
```



In [24]:
```
hour_rate = df.groupby('most ads hour')['converted'].mean().reset_index()

plt.figure(figsize=(12,5))
sns.barplot(data=hour_rate, x= 'most ads hour' , y='converted')
plt.title("Conversion Rate by Hour")
plt.show()
```



6. Statistical Testings

6.1 A/B Test - compare conversion rates

In [26]:
```
control_group = df[df['test group'] == 'psa']
test_group = df[df['test group'] == 'ad']

control_conv = control_group['converted']
test_conv = test_group['converted']

t_stat, p_val = stats.ttest_ind(test_conv , control_conv)

if p_val < 0.05:
    print("Result: Reject H0 -> Significant difference in conversion rates")
else:
    print("Result: Fail to reject H0 -> No significant difference")
```

```
Result: Reject H0 -> Significant difference in conversion rates
```

In [27]:
```
anova_day = stats.f_oneway(
    *[
        df[df["most ads day"] == day]["converted"]
        for day in df["most ads day"].unique()
    ]
)

print("ANOVA - Most Ads Day")
print("F-statistic:", anova_day.statistic)
```

```
print("P-value:", anova_day.pvalue)

if anova_day.pvalue < 0.05:
    print("Result: Reject H0 -> Day significantly affects conversion.")
else:
    print("Result: Fail to reject H0 -> No effect of day.")
```

```
ANOVA - Most Ads Day
F-statistic: 68.38818386898645
P-value: 1.8032007657513903e-85
Result: Reject H0 -> Day significantly affects conversion.
```

In [28]:
```
anova_hour = stats.f_oneway(
    *[
        df[df["most ads hour"] == hr]["converted"]
        for hr in sorted(df["most ads hour"].unique())
    ]
)

print("ANOVA - Most Ads Hour")
print("F-statistic:", anova_hour.statistic)
print("P-value:", anova_hour.pvalue)

if anova_hour.pvalue < 0.05:
    print("Result: Reject H0 -> Hour significantly affects conversion.")
else:
    print("Result: Fail to reject H0.")
```

```
ANOVA - Most Ads Hour
F-statistic: 18.742037312975857
P-value: 7.482025334380472e-77
Result: Reject H0 -> Hour significantly affects conversion.
```

In [ ]:
```

```